

Advanced Algorithms

Klevis Imeri Lecture Notes

Autumn 2026

Contents

1	Approximation Algorithms	1
	Questions	9
2	Approximation schemes	11
	Questions	11
	General questions	13

1 Approximation Algorithms

Definition 1.1 (Problem).

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

Input: $I \in \mathcal{I}$,

Output: $S \in \mathcal{S}_I$.

Π : problem,

$\mathcal{I}$: instance space, I : instance,

$\mathcal{S}$: set of possible outputs, S : output,

$\mathcal{S}_I := \{ S \in \mathcal{S} : S \text{ is valid for } I \}$.

Definition 1.2 (Optimization problem).

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

$$c : \mathcal{I} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}.$$

$$\text{For fixed } I : \quad c(S) := c(I, S), \quad c(\text{OPT}(I)) := c(I, \text{OPT}(I)).$$

$$\text{OPT}(I) \in \mathcal{S}_I := \begin{cases} \arg \min_{T \in \mathcal{S}} c(T), & \text{minimization,} \\ \arg \max_{T \in \mathcal{S}} c(T), & \text{maximization.} \end{cases} + \text{other constraints.}$$

Π : optimization problem,

$\mathcal{I}$: instance space, I : instance,

$\mathcal{S}$: set of possible outputs, S : output,

$\mathcal{S}_I$: optimal solutions for I ,

c : cost function, $c(S)$: cost of S for I ,

$\text{OPT}(I)$: optimal solution for I (assumed to exist).

Definition 1.3 (Algorithm).

$$\mathcal{A} : \mathcal{I} \rightarrow \mathcal{S}, \quad \forall I \in \mathcal{I} : \mathcal{A}(I) \in \mathcal{S}_I.$$

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

Π : problem,

$\mathcal{I}$: instance space, I : instance,

$\mathcal{S}$: set of possible outputs,

$\mathcal{S}_I := \{ S \in \mathcal{S} : S \text{ is valid for } I \}$,

$\mathcal{A}$: algorithm, $\mathcal{A}(I)$: solution returned on I .

Definition 1.4 (Polynomial).

$$\text{poly}(n) := \{ f : \exists c \in \mathbb{N}, f \in O(n^c) \}.$$

$\text{poly}(n)$: functions bounded by a polynomial in n ,

n : input size,

f : function,

c : constant exponent,

$O(n^c)$: asymptotic upper bound of order n^c .

Definition 1.5 (Running time (TODO)).

Definition 1.6 (Efficient algorithm).

$$\exists p \in \text{poly}(n) \forall I \in \mathcal{I} : \text{time}_{\mathcal{A}}(I) \leq p(|I|).$$

$\mathcal{A}$: algorithm,

$\mathcal{I}$: instance space, I : instance,

$|I|$: encoding length of the instance I ,

$\text{time}_{\mathcal{A}}(I)$: running time of $\mathcal{A}$ on I ,

p : polynomial bound, n : input size.

Definition 1.7 (α -approximation).

$$\begin{aligned}
& \mathcal{A} \text{ is an } \alpha\text{-approximation} \iff \\
& \left[\exists p \in \text{poly}(n) \ \forall I \in \mathcal{I} : \text{time}_{\mathcal{A}}(I) \leq p(|I|) \right] \wedge \\
& \quad \forall I \in \mathcal{I} : \\
& \quad \begin{cases} c(\mathcal{A}(I)) \leq \alpha(I) c(\text{OPT}(I)), & \text{minimization,} \\ c(\text{OPT}(I)) \leq \alpha(I) c(\mathcal{A}(I)), & \text{maximization.} \end{cases} \\
& \Pi := (\mathcal{I}, \mathcal{S}), \quad \alpha : \mathcal{I} \rightarrow \mathbb{R}_{\geq 1}. \\
& \mathcal{A} : \mathcal{I} \rightarrow \mathcal{S}, \quad \forall I \in \mathcal{I} : \mathcal{A}(I) \text{ is feasible for } I. \\
& \Pi : \text{optimization problem,} \\
& \mathcal{I} : \text{instance space,} \quad I : \text{instance,} \\
& \mathcal{S} : \text{set of possible outputs,} \\
& \mathcal{S}_I : \text{optimal solutions for } I, \\
& c : \text{cost function,} \\
& \mathcal{A} : \text{algorithm,} \\
& \text{OPT}(I) : \text{optimal solution for } I, \\
& \alpha : \text{approximation-factor function,} \\
& \alpha(I) : \text{approximation factor for } I. \\
& \text{Constant factor } a \geq 1 : \quad \alpha(I) := a \quad (\forall I \in \mathcal{I}).
\end{aligned}$$

Definition 1.8 (Minimum set cover).

$$\Pi_{\text{minimum set cover}} := (\mathcal{I}, \mathcal{S}).$$

$$\mathcal{I} := \left\{ (U, S, w) : 1 \leq |U| < \infty, S \subseteq 2^U, \bigcup_{S_i \in S} S_i = U, w : S \rightarrow \mathbb{R}_{\geq 0} \right\},$$

$$I = (U, S, w) \in \mathcal{I}, \quad c(T) := \sum_{S_i \in T} w(S_i),$$

$$\text{OPT}(I) \in \mathcal{S}_I := \left\{ T \subseteq S : \bigcup_{S_i \in T} S_i = U \wedge T \in \arg \min_{\substack{T' \subseteq S \\ \bigcup_{S_i \in T'} S_i = U}} c(T') \right\}.$$

$\mathcal{I}$: instance space,

$\mathcal{S}$: set of possible outputs,

$\mathcal{S}_I$: minimum-cost covers of U ,

c : cost function,

I : instance,

U : nonempty finite universe,

S : collection of available subsets of U ,

S_i : individual available set,

T : selected subcollection,

$w(S_i)$: cost of set S_i ,

$c(T)$: total cost of the selected sets,

2^U : power set of U .

Example 1.9 (Minimum set cover).

$$U := \{e_1, e_2, e_3, e_4, e_5\}, \quad S := \{S_1, S_2, S_3, S_4\}, \quad I := (U, S, w).$$

i	S_i	$w(S_i) := i^2$
1	$\{e_1, e_2, e_5\}$	1
2	$\{e_4\}$	4
3	$\{e_2, e_3\}$	9
4	$\{e_1, e_4, e_5\}$	16

$$T := \{S_3, S_4\} \subseteq S, \quad \bigcup_{S_i \in T} S_i = U, \quad c(T) = 9 + 16 = 25.$$

$$T^* := \text{OPT}(I) = \{S_1, S_2, S_3\}, \quad c(T^*) = 1 + 4 + 9 = 14.$$

$$|T^*| = 3 > 2 = |T|, \quad c(T^*) = 14 < 25 = c(T).$$

Theorem 1.10 (Complexity of minimum set cover).

$$\Pi_{\text{minimum set cover}} \in \text{NP-hard}.$$

$$\text{SC}_{\text{dec}} := \left\{ (U, S, k) : \begin{array}{l} |U| < \infty, S \subseteq 2^U, k \in \mathbb{N}, \\ \exists T \subseteq S : |T| \leq k, \bigcup_{S_i \in T} S_i = U \end{array} \right\}.$$

$$\text{SC}_{\text{dec}} \in \text{NPC}.$$

Proof (Intuition). A proposed cover can be checked in polynomial time: count its sets and check that their union is the universe. Thus the decision problem is in NP.

Vertex Cover, a known NP-complete problem, asks whether at most k vertices can touch every edge of a graph. Turn each edge into an element of U , and each vertex into the set of edges touching it. Selecting vertices that touch every edge is then equivalent to selecting sets that cover U . This construction takes polynomial time, so Set Cover is NP-hard and hence NP-complete as a decision problem.

With every set cost equal to 1, an optimal cover also tells us whether a cover with at most k sets exists. Finding an optimal cover is therefore NP-hard. $\square$

Definition 1.11 (Greedy set cover algorithm).

$$\mathcal{A}(I) := \text{GreedySetCover}(U, S, w).$$

$$\text{ppi}(S_j) := \begin{cases} \frac{w(S_j)}{|S_j|}, & S_j \neq \emptyset, \\ \infty, & S_j = \emptyset. \end{cases}$$

Algorithm 1 GreedySetCover(U, S, w)

```

1: if  $U = \emptyset$  then
2:   return  $\emptyset$ 
3: else
4:    $S_i \in \arg \min_{S_j \in S} \text{ppi}(S_j)$ 
5:    $U' \leftarrow U \setminus S_i$ 
6:    $S' \leftarrow \{S'_j := S_j \setminus S_i : S_j \in S\}$ 
7:    $w(S'_j) \leftarrow w(S_j) \quad (\forall S'_j \in S')$ 
8:   return  $\{S_i\} \cup \text{GreedySetCover}(U', S', w)$ 
9: end if

```

$$\Pi_{\text{minimum set cover}} = (\mathcal{I}, \mathcal{S}).$$

U : remaining universe,

S : collection of remaining sets,

$w(S_j)$: cost of set S_j ,

$\text{ppi}(S_j)$: cost per remaining element of S_j ,

S_i : set chosen in the current call,

U' : universe after removing the elements of S_i ,

S'_j : remaining elements of S_j ,

S' : collection of sets for the recursive call.

Example 1.12 (Greedy set cover).

$$U_0 := \{e_1, e_2, e_3, e_4, e_5\}, \quad S := \{S_1, S_2, S_3, S_4\}.$$

$$\begin{aligned} S_1 &= \{e_1, e_2, e_5\}, & S_2 &= \{e_4\}, \\ S_3 &= \{e_2, e_3\}, & S_4 &= \{e_1, e_4, e_5\}. \end{aligned}$$

$$w(S_j) := j^2 \quad (j \in \{1, 2, 3, 4\}), \quad I := (U_0, S, w).$$

$$S_j^{(t)} := S_j \cap U_t, \quad S^{(t)} := \{S_j^{(t)} : j \in \{1, 2, 3, 4\}\}.$$

$$\text{ppi}_{j,t} := \text{ppi}(S_j^{(t)}) = \begin{cases} \frac{w(S_j)}{|S_j^{(t)}|}, & S_j^{(t)} \neq \emptyset, \\ \infty, & S_j^{(t)} = \emptyset. \end{cases}$$

$$i_t \in \arg \min_{j \in \{1, 2, 3, 4\}} \text{ppi}_{j,t}, \quad U_{t+1} := U_t \setminus S_{i_t}.$$

t	U_t	$\text{ppi}_{1,t}$	$\text{ppi}_{2,t}$	$\text{ppi}_{3,t}$	$\text{ppi}_{4,t}$	S_{i_t}	U_{t+1}
0	U_0	$\frac{1}{3}$	4	$\frac{9}{2}$	$\frac{16}{3}$	S_1	$\{e_3, e_4\}$
1	$\{e_3, e_4\}$	∞	4	9	16	S_2	$\{e_3\}$
2	$\{e_3\}$	∞	∞	9	∞	S_3	$\emptyset$

$$\text{GreedySetCover}(U_3, S^{(3)}, w) = \emptyset.$$

$$\begin{aligned} \mathcal{A}(I) &= \text{GreedySetCover}(U_0, S^{(0)}, w) \\ &= \{S_1\} \cup (\{S_2\} \cup (\{S_3\} \cup \emptyset)) \\ &= \{S_1, S_2, S_3\}, \end{aligned}$$

$$c(\mathcal{A}(I)) = 1 + 4 + 9 = 14.$$

Theorem 1.13 (Termination, feasibility, and efficiency of greedy set cover).

$\forall I = (U, S, w) \in \mathcal{I} : \text{GreedySetCover}(U, S, w) \text{ terminates, is feasible, and is efficient.}$

Proof.

$I = (U, S, w) \in \mathcal{I}, \quad \mathcal{A}(I) := \text{GreedySetCover}(U, S, w),$

$U_t : \text{remaining universe after } t \text{ selections,} \quad U_0 := U.$

1. Termination: $\mathcal{A}(I) \text{ terminates} \iff \exists r \in \mathbb{N}_{\geq 0} : U_r = \emptyset.$

$i_t : \text{index of the greedy set selected at step } t,$

$U_t \neq \emptyset \implies |S_{i_t} \cap U_t| \geq 1,$

$U_{t+1} := U_t \setminus S_{i_t}, \quad |U_{t+1}| = |U_t| - |S_{i_t} \cap U_t| \leq |U_t| - 1$

$\implies \exists r \leq |U| : U_r = \emptyset.$

2. Feasibility: $\mathcal{A}(I) \text{ is feasible} \iff \mathcal{A}(I) \subseteq S \wedge \bigcup_{S_j \in \mathcal{A}(I)} S_j = U.$

$T_t := \{S_{i_s} : 0 \leq s < t\}, \quad T_0 = \emptyset, \quad T_{t+1} = T_t \cup \{S_{i_t}\},$

$T_t \subseteq S \wedge U_t = U \setminus \bigcup_{S_j \in T_t} S_j \quad (\text{induction on } t),$

$\mathcal{A}(I) = T_r \subseteq S \wedge \emptyset = U_r = U \setminus \bigcup_{S_j \in T_r} S_j \implies \bigcup_{S_j \in \mathcal{A}(I)} S_j = U.$

3. Efficiency: $\mathcal{A} \text{ is efficient} \iff \exists q \text{ polynomial } \forall I \in \mathcal{I} : \text{time}_{\mathcal{A}}(I) \leq q(|I|).$

$\# \text{ operations per call} \in O(|S| |U|), \quad \# \text{ recursive calls} = r + 1 \leq |U| + 1,$

$|U| \leq |I| \wedge |S| \leq |I| \implies \# \text{ operations} \in O(|S| |U|^2) \subseteq O(|I|^3),$

$\text{time}_{\text{one operation}} \in \text{poly}(|I|) \implies \text{time}_{\mathcal{A}}(I) \in \text{poly}(|I|).$

□

Definition 1.14 (Harmonic number).

$$H_0 := 0, \quad H_n := \sum_{\ell=1}^n \frac{1}{\ell} \quad (n \in \mathbb{N}_{\geq 1}).$$

$H_n : \text{the } n\text{-th harmonic number.}$

Lemma 1.15 (Price of the first greedy set).

$I = (U, S, w) \in \mathcal{I}, \quad n := |U|,$

$\text{OPT}^* := \text{OPT}(I), \quad \text{OPT} := \text{OPT}^* \setminus \{\emptyset\}, \quad c(\text{OPT}) = c(\text{OPT}^*),$

$S_i \in \arg \min_{S_j \in S} \text{ppi}(S_j) \implies \text{ppi}(S_i) \leq \frac{c(\text{OPT})}{n}.$

Proof.

$\bigcup_{O_j \in \text{OPT}} O_j = U \implies \sum_{O_j \in \text{OPT}} |O_j| \geq n.$

$c(\text{OPT}) = \sum_{O_j \in \text{OPT}} w(O_j) = \sum_{O_j \in \text{OPT}} |O_j| \text{ppi}(O_j).$

$\text{ppi}(S_i) = \min_{S_j \in S} \text{ppi}(S_j) \leq \min_{O_j \in \text{OPT}} \text{ppi}(O_j) \leq \frac{\sum_{O_j \in \text{OPT}} |O_j| \text{ppi}(O_j)}{\sum_{O_j \in \text{OPT}} |O_j|} = \frac{c(\text{OPT})}{\sum_{O_j \in \text{OPT}} |O_j|} \leq \frac{c(\text{OPT})}{n}.$

□

Theorem 1.16 (Approximation ratio of greedy set cover).

GreedySetCover is an H_n -approximation algorithm.

$$n := |U|.$$

Proof. GreedySetCover is an H_n -approximation algorithm $\stackrel{\text{Def. 1.7, Thm. 1.13}}{\iff} \forall I \in \mathcal{I} : c(\mathcal{A}(I)) \leq H_n c(\text{OPT}(I))$.

Fix arbitrary $I = (U, S, w) \in \mathcal{I}$.

Strong induction on n .

1. Base case: $n = 1 \implies \forall S_j \in S : S_j \neq \emptyset \implies S_j = U$,

$$c(\mathcal{A}(I)) = \min_{S_j \in S : S_j \neq \emptyset} w(S_j) = c(\text{OPT}(I)) = H_1 c(\text{OPT}(I)).$$

2. Inductive step: $n \geq 2$.

$$S_i \in \arg \min_{S_j \in S} \text{ppi}(S_j) \quad (\text{first greedy set}),$$

$$k := |S_i| \geq 1, \quad U' := U \setminus S_i, \quad |U'| = n - k,$$

$$S' := \{ S_j := S_j \setminus S_i : S_j \in S \}, \quad w(S'_j) := w(S_j), \quad I' := (U', S', w),$$

$$C' := c(\text{GreedySetCover}(U', S', w)).$$

$$c(\mathcal{A}(I)) = w(S_i) + C'$$

$$\stackrel{\text{Def. of ppi}}{=} k \text{ ppi}(S_i) + C'$$

$$\leq k \text{ ppi}(S_i) + H_{n-k} c(\text{OPT})$$

$$\stackrel{\text{Lemma 1.15}}{\leq} k \frac{c(\text{OPT})}{n} + H_{n-k} c(\text{OPT}) = \left(\frac{k}{n} + H_{n-k} \right) c(\text{OPT}) \leq H_n c(\text{OPT}).$$

$$n = k \implies C' = 0,$$

$$n > k : \underbrace{C' \leq H_{n-k} c(\text{OPT}(I'))}_{\text{IH}} \leq H_{n-k} c(\text{OPT}).$$

$$\frac{k}{n} = \sum_{\ell=n-k+1}^n \frac{1}{n} \leq \sum_{\ell=n-k+1}^n \frac{1}{\ell} = H_n - H_{n-k}.$$

IH : induction hypothesis.

□

Example 1.17 (Tight bound for greedy set cover).

$$k \in \mathbb{N}_{\geq 2}, \quad \forall i \in \{1, \dots, k\} : |A_i| = |B_i| = 2^{i-1}, \quad S_i := A_i \cup B_i,$$

$$\forall i : A_i \cap B_i = \emptyset, \quad \forall i \neq j : S_i \cap S_j = \emptyset,$$

$$U := \bigcup_{i=1}^k S_i, \quad n := |U| = \sum_{i=1}^k 2^i = 2(2^k - 1).$$

$$S_{k+1} := \bigcup_{i=1}^k A_i, \quad S_{k+2} := \bigcup_{i=1}^k B_i,$$

$$S_{k+1} \cap S_{k+2} = \emptyset \wedge S_{k+1} \cup S_{k+2} = U,$$

$$S := \{S_1, \dots, S_{k+2}\}, \quad \forall S_i \in S : w(S_i) := 1, \quad I_k := (U, S, w).$$

$$j \in \{1, \dots, k\}, \quad U_j := \bigcup_{i=1}^j S_i, \quad |U_j| = 2(2^j - 1),$$

$$|S_j \cap U_j| = 2^j, \quad |S_{k+1} \cap U_j| = |S_{k+2} \cap U_j| = \sum_{i=1}^j 2^{i-1} = 2^j - 1,$$

$$\forall i \leq j : \text{ppi}(S_j \cap U_j) = \frac{1}{2^j} \leq \frac{1}{2^i} = \text{ppi}(S_i \cap U_j),$$

$$\text{ppi}(S_j \cap U_j) = \frac{1}{2^j} < \frac{1}{2^j - 1} = \text{ppi}(S_{k+1} \cap U_j) = \text{ppi}(S_{k+2} \cap U_j),$$

$$\forall i > j, i \leq k : S_i \cap U_j = \emptyset \implies \text{ppi}(S_i \cap U_j) = \infty.$$

$$S_k \longrightarrow S_{k-1} \longrightarrow \dots \longrightarrow S_1,$$

$$\mathcal{A}(I_k) = \{S_1, \dots, S_k\}, \quad c(\mathcal{A}(I_k)) = k.$$

$$\max_{S_i \in S} |S_i| = 2^k < 2(2^k - 1) = n \implies c(\text{OPT}(I_k)) \geq 2,$$

$$S_{k+1} \cup S_{k+2} = U \wedge c(\{S_{k+1}, S_{k+2}\}) = 2 \implies c(\text{OPT}(I_k)) = 2.$$

$$\frac{c(\mathcal{A}(I_k))}{c(\text{OPT}(I_k))} = \frac{k}{2} = \frac{1}{2} \log_2 \left(\frac{n}{2} + 1 \right) = \Theta(\log n) = \Theta(H_n).$$

Theorem 1.18 (Hardness of approximating set cover (Dinur–Steurer, 2014)).

$$\forall \varepsilon \in (0, 1) : (1 - \varepsilon) \ln |U| \text{-approximation of } \Pi_{\text{minimum set cover}} \text{ is NP-hard.}$$

Proof.

□

Questions

1. Should Definition 1.1 explicitly define $\alpha : \mathcal{I} \rightarrow \mathbb{R}_{\geq 1}$ and use $\alpha(I)$ in the approximation inequalities, allowing instance-dependent factors such as $\alpha(I) = H_{|U|}$ and constant factors as constant functions $\alpha(I) = a$ for all I ?
2. Should an α -approximation algorithm be required to be efficient, i.e., run in polynomial time? The introduction of the lecture notes describes approximation algorithms as efficient, but Definition 1.1 only states the approximation inequality.
3. Should the minimum set cover problem require $|U| \geq 1$ so that the H_n -approximation guarantee always has $H_n \geq 1$? The proof of Theorem 1.3 in the lecture notes uses $n = 0$ as its base case, but $H_0 = 0$, whereas approximation factors are normally required to satisfy $\alpha \geq 1$. If the empty universe is allowed, how should the approximation factor be stated? Does Definition 1.1 allow α to depend on n , with $\alpha(0) = 1$ and $\alpha(n) = H_n$ for $n \geq 1$, or

should the empty case be handled separately from the approximation guarantee?

2 Approximation schemes

Definition 2.1 (Polynomial time approximation scheme (PTAS)).

$\mathcal{A}$ is a PTAS for $\Pi_{\text{optimization}}$ $\iff$

$\forall \varepsilon > 0 \exists q_\varepsilon$ polynomial $\forall I \in \mathcal{I}$:

$$\begin{aligned} & \mathcal{A}_\varepsilon(I) \text{ terminates and is feasible for } I \wedge \\ & \begin{cases} c(\mathcal{A}_\varepsilon(I)) \leq (1 + \varepsilon)c(\text{OPT}(I)), & \text{minimization,} \\ c(\mathcal{A}_\varepsilon(I)) \geq (1 - \varepsilon)c(\text{OPT}(I)), & \text{maximization} \end{cases} \wedge \\ & \text{time}_{\mathcal{A}_\varepsilon}(I) \leq q_\varepsilon(|I|). \end{aligned}$$

$$\begin{aligned} \Pi_{\text{optimization}} &:= (\mathcal{I}, \mathcal{S}), \quad \text{OPT}(I) \in \mathcal{S}_I. \\ \forall \varepsilon > 0 : \quad \mathcal{A}_\varepsilon : \mathcal{I} &\rightarrow \mathcal{S}. \end{aligned}$$

ε : approximation parameter (fixed),

$\mathcal{A}_\varepsilon$: algorithm for fixed ε ,

q_ε : polynomial in $|I|$ whose coefficients and degree may depend on ε .

Definition 2.2 (Fully polynomial time approximation scheme (FPTAS)).

$\mathcal{A}$ is an FPTAS for $\Pi_{\text{optimization}}$ $\iff$

$\exists q$ polynomial $\forall \varepsilon > 0 \forall I \in \mathcal{I}$:

$$\begin{aligned} & \mathcal{A}(I, \varepsilon) \text{ terminates and is feasible for } I \wedge \\ & \begin{cases} c(\mathcal{A}(I, \varepsilon)) \leq (1 + \varepsilon)c(\text{OPT}(I)), & \text{minimization,} \\ c(\mathcal{A}(I, \varepsilon)) \geq (1 - \varepsilon)c(\text{OPT}(I)), & \text{maximization} \end{cases} \wedge \\ & \text{time}_{\mathcal{A}}(I, \varepsilon) \leq q\left(|I|, \frac{1}{\varepsilon}\right). \end{aligned}$$

$$\begin{aligned} \Pi_{\text{optimization}} &:= (\mathcal{I}, \mathcal{S}), \quad \text{OPT}(I) \in \mathcal{S}_I. \\ \mathcal{A} : \mathcal{I} \times \mathbb{R}_{>0} &\rightarrow \mathcal{S}. \end{aligned}$$

ε : approximation parameter (input),

q : one fixed polynomial in $|I|$ and $1/\varepsilon$.

Example 2.3 (PTAS versus FPTAS running time).

$$\text{time}_{\mathcal{A}_\varepsilon}(I) \in O(|I|^{1/\varepsilon}).$$

$$\varepsilon > 0 \text{ fixed} : \quad |I|^{1/\varepsilon} \in \text{poly}(|I|) \implies \text{PTAS running-time bound.}$$

$$|I| = 2 : \quad |I|^{1/\varepsilon} = 2^{1/\varepsilon} \notin \text{poly}(1/\varepsilon) \implies \text{not an FPTAS running-time bound.}$$

Questions

1. The introduction to Chapter 2 states that the greedy algorithms from Chapter 1 approximate the optimum within a constant factor, including minimum set cover. Is this a typo? The guarantee for greedy set cover is $H_n = \Theta(\log n)$, where $n = |U|$, so its approximation

- factor is not constant. Should the statement distinguish this logarithmic factor from the constant factors for matching and vertex cover?
2. For maximization, should the PTAS and FPTAS guarantee be defined as $c(\mathcal{A}(I, \varepsilon)) \geq (1 - \varepsilon)c(\text{OPT}(I))$? How does this relate to Definition 1.1, which uses $c(\text{OPT}(I)) \leq \alpha c(\mathcal{A}(I))$? Is the intended correspondence $\alpha = 1/(1 - \varepsilon)$ for $0 < \varepsilon < 1$, rather than $\alpha = 1 + \varepsilon$?

General questions

1. When the lecture notes state a theorem but refer to a paper or another source for its proof, are we expected to study and be able to reproduce that proof, or is it sufficient to know the theorem's statement and how to use it?